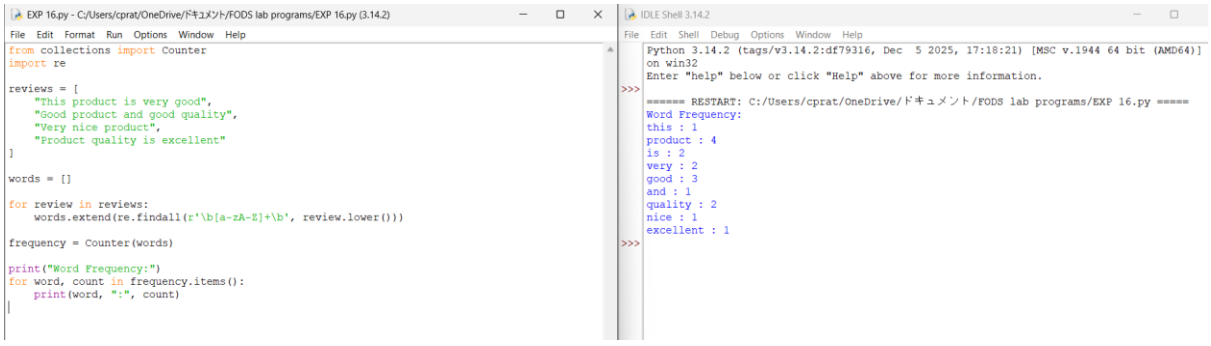
# LAB PROGRAMS SET (16 – 30)

## 16. Word Frequency Distribution in Customer Reviews

**Python code :**

```
from collections import Counter
import re
reviews = [
    "This product is very good",
    "Good product and good quality",
    "Very nice product",
    "Product quality is excellent"
]
words = []
for review in reviews:
    words.extend(re.findall(r'\b[a-zA-Z]+\b', review.lower()))
frequency = Counter(words)
print("Word Frequency:")
for word, count in frequency.items():
    print(word, ":", count)
```

**Output :**




## 17. Word Frequency with CSV, Stop Words and Bar Graph

**Python code:**

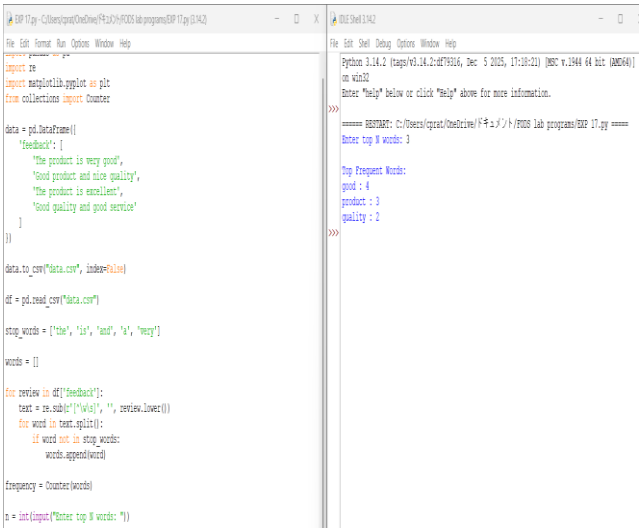
```
import pandas as pd
import re
```

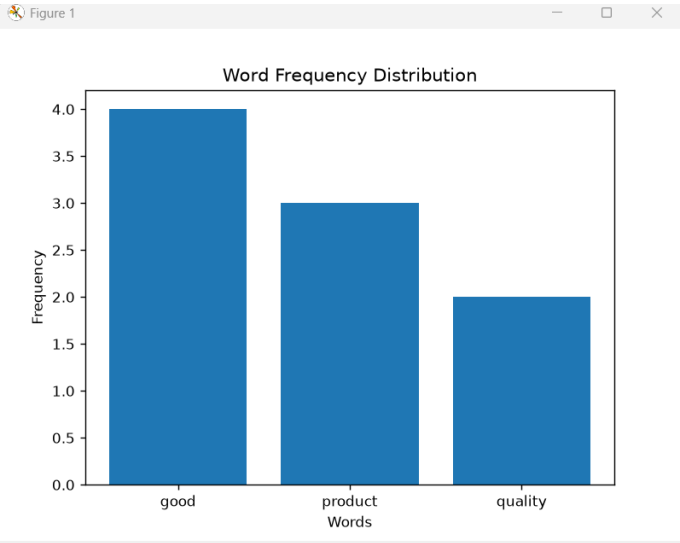
```
import matplotlib.pyplot as plt
from collections import Counter
data = pd.DataFrame({
    'feedback': [
        'The product is very good',
        'Good product and nice quality',
        'The product is excellent',
        'Good quality and good service'
    ]
})
data.to_csv("data.csv", index=False)
df = pd.read_csv("data.csv")
stop_words = ['the', 'is', 'and', 'a', 'very']
words = []
for review in df['feedback']:
    text = re.sub(r'[^\w\s]', '', review.lower())
    for word in text.split():
        if word not in stop_words:
            words.append(word)
frequency = Counter(words)
n = int(input("Enter top N words: "))
top_words = frequency.most_common(n)
print("\nTop Frequent Words:")
for word, count in top_words:
    print(word, ":", count)
words_list = [x[0] for x in top_words]
counts = [x[1] for x in top_words]
plt.bar(words_list, counts)
plt.xlabel("Words")
plt.ylabel("Frequency")
```

plt.title("Word Frequency Distribution")

plt.show()

**Output :**




**18. Mean, Median, Standard Deviation, Boxplot, Scatter Plot and Q-Q Plot**

**Python code:**

import pandas as pd

import matplotlib.pyplot as plt

import scipy.stats as stats

data = {

'Age': [23, 25, 27, 29, 31, 33, 35, 37, 39,

41, 43, 45, 47, 49, 51, 53, 55, 57],

'Fat': [15, 18, 20, 22, 24, 26, 28, 30, 32,

34, 36, 38, 40, 42, 44, 46, 48, 50]

}

df = pd.DataFrame(data)

print("Mean:")

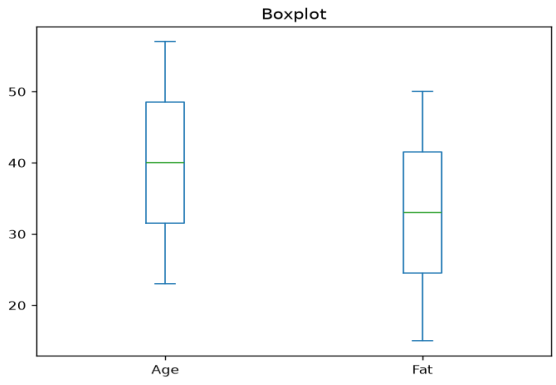
print(df.mean())

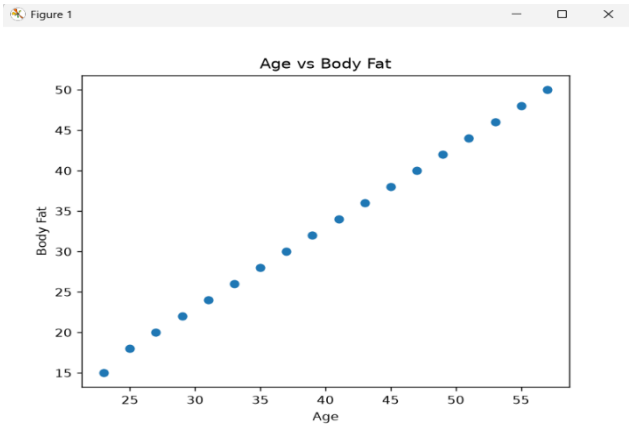
print("\nMedian:")

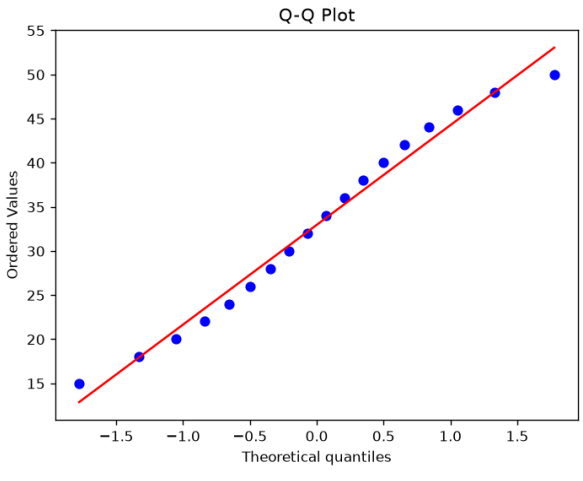
print(df.median())

```
print("\nStandard Deviation:")
print(df.std())
# Boxplot
df.plot(kind='box')
plt.title("Boxplot")
plt.show()
# Scatter plot
plt.scatter(df['Age'], df['Fat'])
plt.xlabel("Age")
plt.ylabel("Body Fat")
plt.title("Age vs Body Fat")
plt.show()
# Q-Q plot
stats.probplot(df['Fat'], dist="norm", plot=plt)
plt.title("Q-Q Plot")
plt.show()
```

**Output :**



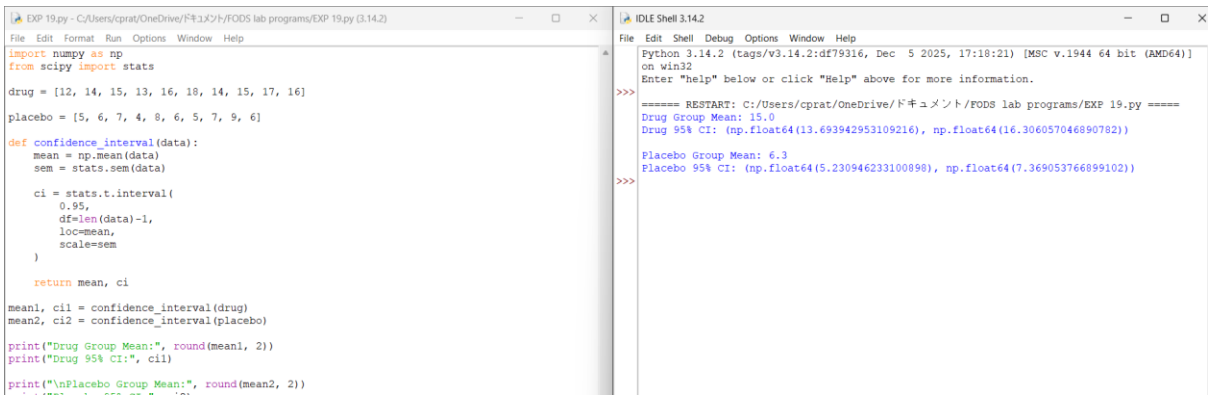



**19. 95% Confidence Interval for Drug and Placebo**

**Python code:**

```
import numpy as np
from scipy import stats
drug = [12, 14, 15, 13, 16, 18, 14, 15, 17, 16]
placebo = [5, 6, 7, 4, 8, 6, 5, 7, 9, 6]
def confidence_interval(data):
    mean = np.mean(data)
    sem = stats.sem(data)
    ci = stats.t.interval(
        0.95,
        df=len(data)-1,
        loc=mean,
        scale=sem
    )
    return mean, ci
mean1, ci1 = confidence_interval(drug)
mean2, ci2 = confidence_interval(placebo)
print("Drug Group Mean:", round(mean1, 2))
print("Drug 95% CI:", ci1)
print("\nPlacebo Group Mean:", round(mean2, 2))
print("Placebo 95% CI:", ci2)
```

**Output :**

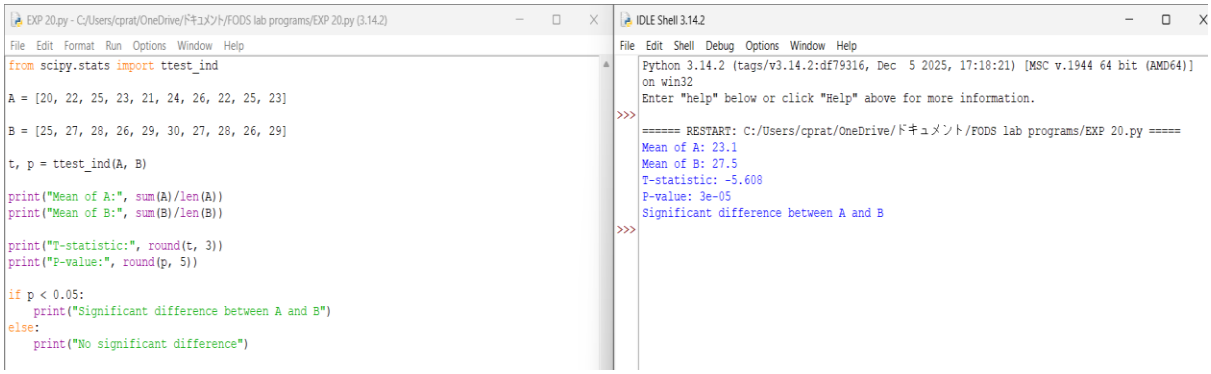


## 20. A/B Testing Using Statistical Significance

**Python code:**

```
from scipy.stats import ttest_ind
A = [20, 22, 25, 23, 21, 24, 26, 22, 25, 23]
B = [25, 27, 28, 26, 29, 30, 27, 28, 26, 29]
t, p = ttest_ind(A, B)
print("Mean of A:", sum(A)/len(A))
print("Mean of B:", sum(B)/len(B))
print("T-statistic:", round(t, 3))
print("P-value:", round(p, 5))
if p < 0.05:
    print("Significant difference between A and B")
else:
    print("No significant difference")
```

**Output :**



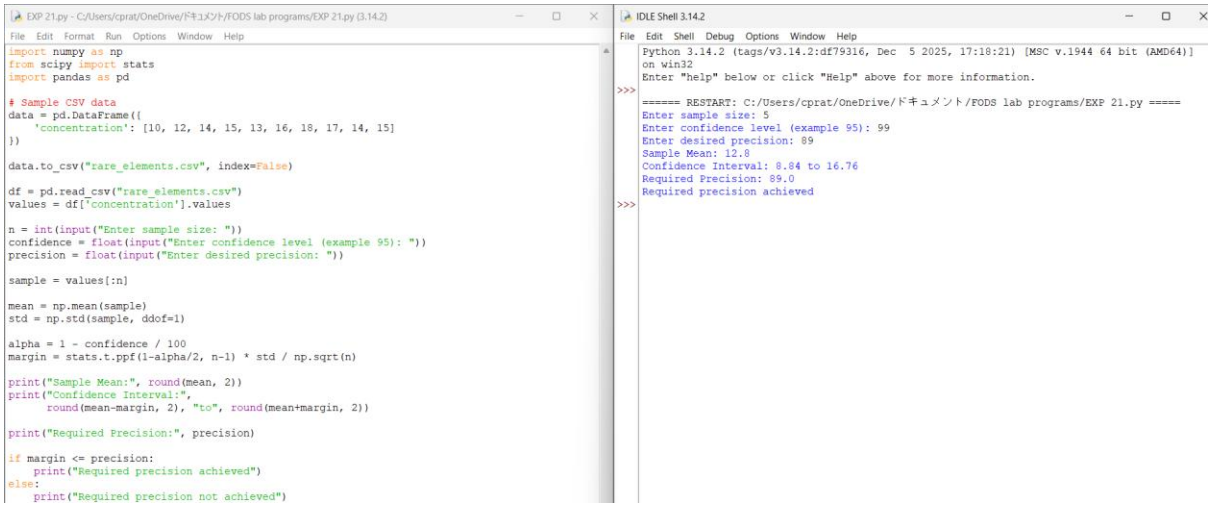

## 21. Point Estimation and Confidence Interval Using NumPy

**Python code:**

```
import numpy as np
from scipy import stats
import pandas as pd
data = pd.DataFrame({
    'concentration': [10, 12, 14, 15, 13, 16, 18, 17, 14, 15]
})
```

```
data.to_csv("rare_elements.csv", index=False)
df = pd.read_csv("rare_elements.csv")
values = df['concentration'].values
n = int(input("Enter sample size: "))
confidence = float(input("Enter confidence level (example 95): "))
precision = float(input("Enter desired precision: "))
sample = values[:n]
mean = np.mean(sample)
std = np.std(sample, ddof=1)
alpha = 1 - confidence / 100
margin = stats.t.ppf(1-alpha/2, n-1) * std / np.sqrt(n)
print("Sample Mean:", round(mean, 2))
print("Confidence Interval:",
      round(mean-margin, 2), "to", round(mean+margin, 2))
print("Required Precision:", precision)
if margin <= precision:
    print("Required precision achieved")
else:
    print("Required precision not achieved")
```

**Output :**

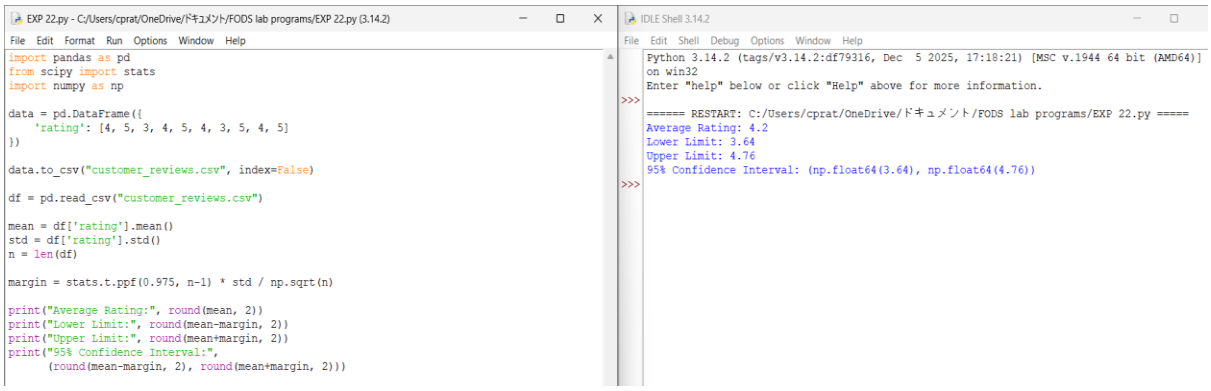


## 22. Confidence Interval for Customer Ratings Using Pandas

**Python code:**

```
import pandas as pd
from scipy import stats
import numpy as np
data = pd.DataFrame({
    'rating': [4, 5, 3, 4, 5, 4, 3, 5, 4, 5]
})
data.to_csv("customer_reviews.csv", index=False)
df = pd.read_csv("customer_reviews.csv")
mean = df['rating'].mean()
std = df['rating'].std()
n = len(df)
margin = stats.t.ppf(0.975, n-1) * std / np.sqrt(n)
print("Average Rating:", round(mean, 2))
print("Lower Limit:", round(mean-margin, 2))
print("Upper Limit:", round(mean+margin, 2))
print("95% Confidence Interval:",
      (round(mean-margin, 2), round(mean+margin, 2)))
```

**Output :**



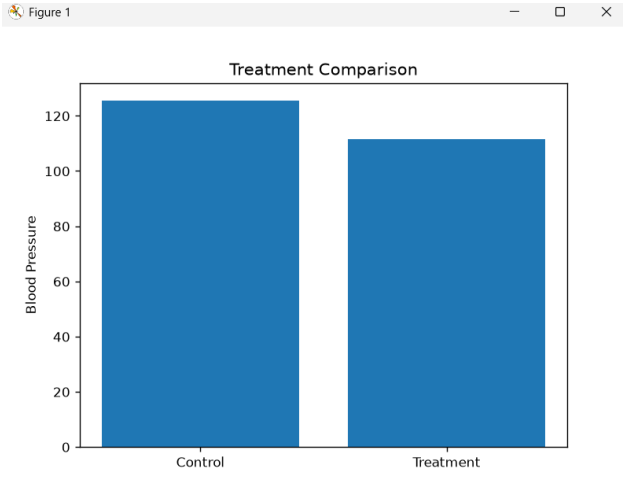

## 23. Hypothesis Testing and P-value Visualization

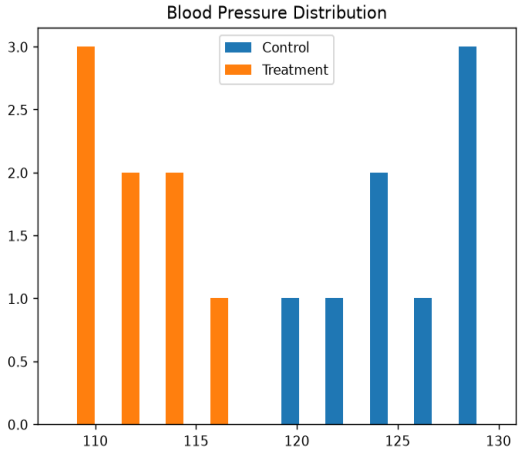
**Python code:**

```
import numpy as np
import matplotlib.pyplot as plt
```

```
from scipy.stats import ttest_ind
control = [120, 125, 130, 128, 122, 126, 124, 129]
treatment = [110, 112, 115, 108, 113, 111, 109, 114]
t, p = ttest_ind(control, treatment)
print("T-statistic:", round(t, 3))
print("P-value:", round(p, 5))
if p < 0.05:
    print("Treatment has a significant effect")
else:
    print("Treatment has no significant effect")
plt.bar(['Control', 'Treatment'],
        [np.mean(control), np.mean(treatment)])
plt.ylabel("Blood Pressure")
plt.title("Treatment Comparison")
plt.show()
plt.hist([control, treatment],
         label=['Control', 'Treatment'])
plt.legend()
plt.title("Blood Pressure Distribution")
plt.show()
```

**Output :**



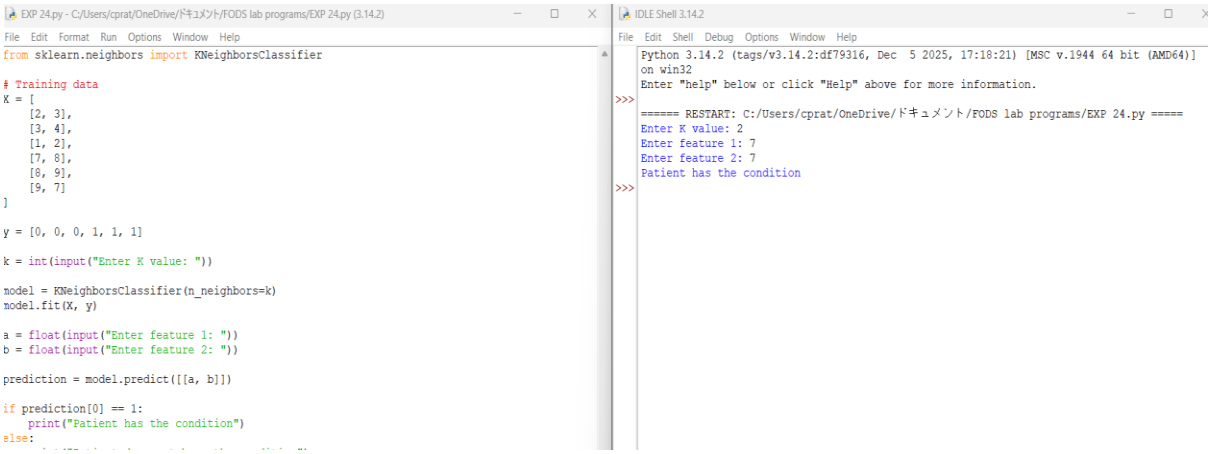



## 24. K-Nearest Neighbors (KNN) Classifier

**Python code:**

```
from sklearn.neighbors import KNeighborsClassifier
X = [
    [2, 3],
    [3, 4],
    [1, 2],
    [7, 8],
    [8, 9],
    [9, 7]
]
y = [0, 0, 0, 1, 1, 1]
k = int(input("Enter K value: "))
model = KNeighborsClassifier(n_neighbors=k)
model.fit(X, y)
a = float(input("Enter feature 1: "))
b = float(input("Enter feature 2: "))
prediction = model.predict([[a, b]])
if prediction[0] == 1:
    print("Patient has the condition")
else:
    print("Patient does not have the condition")
```

**Ouput :**

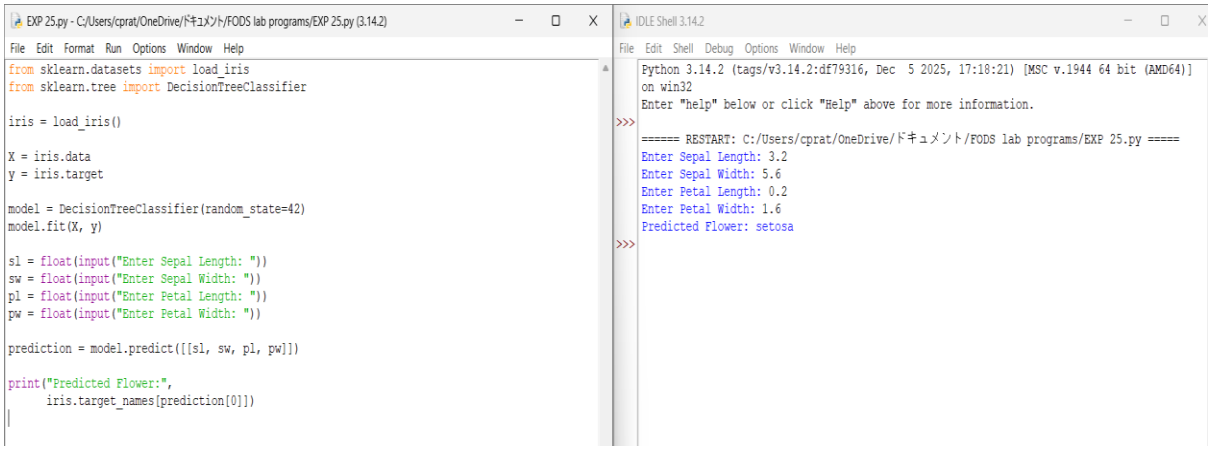


**25. Decision Tree for Iris Flower Classification**

**Python code:**

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
iris = load_iris()
X = iris.data
y = iris.target
model = DecisionTreeClassifier(random_state=42)
model.fit(X, y)
sl = float(input("Enter Sepal Length: "))
sw = float(input("Enter Sepal Width: "))
pl = float(input("Enter Petal Length: "))
pw = float(input("Enter Petal Width: "))
prediction = model.predict([[sl, sw, pl, pw]])
print("Predicted Flower:",
      iris.target_names[prediction[0]])
```

**Output :**




**26. Linear Regression for Housing Price Prediction**

**Python code:**

```
import numpy as np
from sklearn.linear_model import LinearRegression
X = np.array([[1000], [1500], [2000], [2500], [3000]])
y = np.array([200000, 300000, 400000, 500000, 600000])
```

model = LinearRegression()

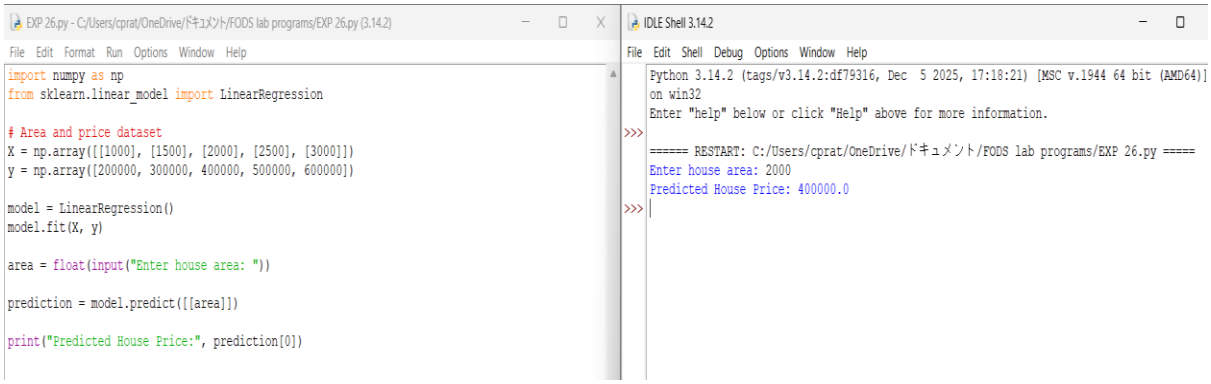
model.fit(X, y)

area = float(input("Enter house area: "))

prediction = model.predict([[area]])

print("Predicted House Price:", prediction[0])

**Output :**




**27. Logistic Regression for Customer Churn Prediction**

**Python code:**

from sklearn.linear_model import LogisticRegression

X = [

[100, 12],

[200, 24],

[150, 18],

[50, 3],

[60, 4],

[80, 5],

[250, 30],

[70, 6]

]

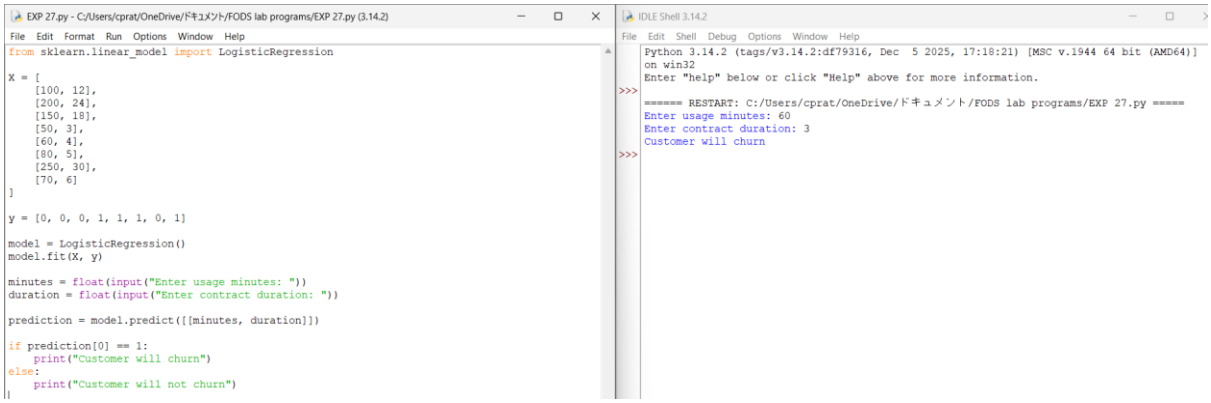
y = [0, 0, 0, 1, 1, 1, 0, 1]

model = LogisticRegression()

model.fit(X, y)

```
minutes = float(input("Enter usage minutes: "))
duration = float(input("Enter contract duration: "))
prediction = model.predict([[minutes, duration]])
if prediction[0] == 1:
    print("Customer will churn")
else:
    print("Customer will not churn")
```

**Output :**




**28. K-Means Clustering for Customer Segmentation**

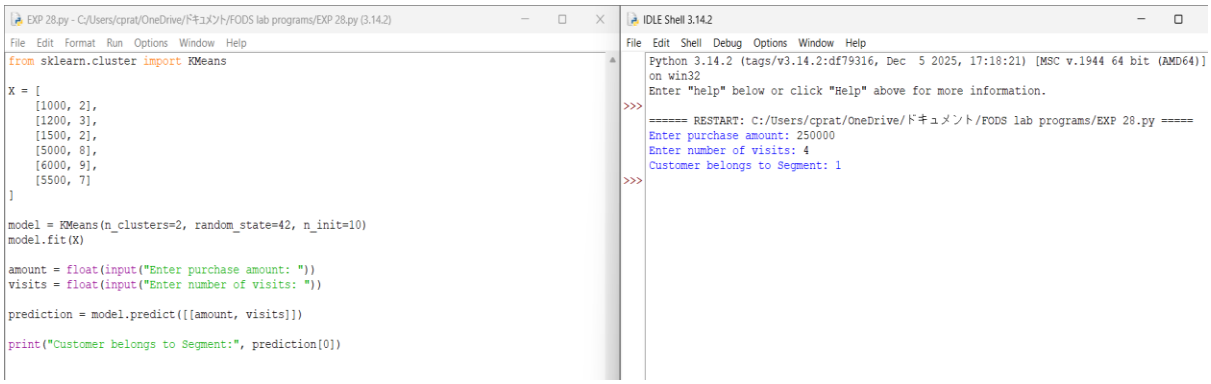
**Python code:**

```
from sklearn.cluster import KMeans
X = [
    [1000, 2],
    [1200, 3],
    [1500, 2],
    [5000, 8],
    [6000, 9],
    [5500, 7]
]
model = KMeans(n_clusters=2, random_state=42, n_init=10)
model.fit(X)
amount = float(input("Enter purchase amount: "))
visits = float(input("Enter number of visits: "))
```

prediction = model.predict([[amount, visits]])

print("Customer belongs to Segment:", prediction[0])

**Output :**




**29. Evaluation Metrics for Model Performance**

**Python code:**

from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeClassifier

from sklearn.metrics import accuracy_score, precision_score

from sklearn.metrics import recall_score, f1_score

iris = load_iris()

X = iris.data

y = iris.target

features = input("Enter features (sepal length, sepal width, petal length, petal width): ")

target = input("Enter target variable: ")

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.2, random_state=42

)

model = DecisionTreeClassifier(random_state=42)

model.fit(X_train, y_train)

pred = model.predict(X_test)

print("Target Variable:", target)

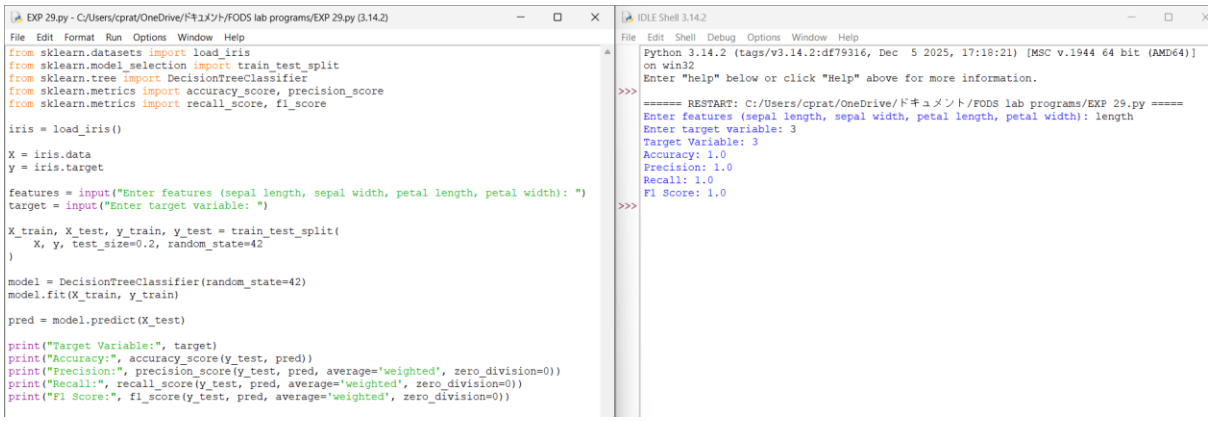
print("Accuracy:", accuracy_score(y_test, pred))

print("Precision:", precision_score(y_test, pred, average='weighted', zero_division=0))

print("Recall:", recall_score(y_test, pred, average='weighted', zero_division=0))

print("F1 Score:", f1_score(y_test, pred, average='weighted', zero_division=0))

**Output :**




**30. CART for Car Price Prediction**

**Python code:**

import numpy as np

from sklearn.tree import DecisionTreeRegressor, export_text

X = np.array([

[20000, 2],

[30000, 3],

[40000, 4],

[50000, 5],

[60000, 6],

[70000, 7]

])

y = [800000, 700000, 600000, 500000, 400000, 300000]

model = DecisionTreeRegressor(random_state=42)

model.fit(X, y)

mileage = float(input("Enter car mileage: "))

age = float(input("Enter car age: "))

prediction = model.predict([[mileage, age]])

print("Predicted Car Price:", prediction[0])

print("\nDecision Path:")

print(export_text(model, feature_names=['Mileage', 'Age']))

**Output :**

